Compte-rendu de TP : Programmation Sockets

TP - Réseaux

Mohamed Ali BENBELGHITH - GL2/1

30 avril 2026

Table des matières

1 Introduction **2**

2 Partie 2 : Développement d'un client HTTP **2**
2.1 Objectif 2
2.2 Code (Extrait) 2
2.3 Observations et Réponses 2

3 Partie 3 : Transfert de messages en mode connecté (TCP) **3**
3.1 Objectif 3
3.2 Résultats d'exécution 3
3.3 Questions théoriques 3

4 Partie 4 : Transfert de messages en mode UDP **3**
4.1 Comparaison TCP vs UDP 3

5 Partie 5 : Serveur en mode concurrent **4**
5.1 Architecture 4
5.2 Services proposés 4

6 Conclusion **4**

1 Introduction

L'objectif de ce TP est de se familiariser avec l'interface de programmation **Berkeley Sockets** pour les communications réseau. Nous explorons les protocoles TCP (mode connecté) et UDP (mode non connecté), ainsi que le développement d'un serveur concurrent capable de gérer plusieurs services et plusieurs clients simultanément.

2 Partie 2 : Développement d'un client HTTP

2.1 Objectif

Développer un client TCP simple capable d'envoyer des requêtes textuelles conformes au protocole HTTP/1.1 à un serveur web.

2.2 Code (Extrait)

```c
// Connexion au serveur
if (connect(sockfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0)
    perror("ERROR connecting");

// Lecture de la requete clavier et envoi
printf("Entrez votre requete HTTP :\n");
fgets(buffer, 1024, stdin);
send(sockfd, buffer, strlen(buffer), 0);

// Reception de la reponse
while ((n = recv(sockfd, buffer, 1023, 0)) > 0) {
    buffer[n] = '\0';
    printf("%s", buffer);
}
```

2.3 Observations et Réponses

- **Handshake TCP :** La connexion s'établit via un cycle SYN, SYN-ACK, ACK.
- **Fermeture :** Le serveur ferme généralement la connexion après avoir envoyé la réponse (Connection : close).
- **Ports :** Le serveur utilise le port fixe 80, tandis que le client utilise un port éphémère aléatoire choisi par l'OS.

3 Partie 3 : Transfert de messages en mode connecté (TCP)

3.1 Objectif

Envoyer 60 fois l'heure courante du serveur vers le client en utilisant le protocole TCP.

3.2 Résultats d'exécution

Lors de l'exécution, nous observons que TCP garantit l'ordre et la fiabilité.

```
[SERVEUR] Client connecte depuis 127.0.0.1:54321
[SERVEUR] Message 1 envoye : Il est 14:05:01 !
[SERVEUR] Message 2 envoye : Il est 14:05:02 !
...
```

3.3 Questions théoriques

- **Nagle :** Sans `sleep(1)`, TCP peut regrouper plusieurs petits messages `send()` dans un seul segment réseau pour optimiser la bande passante.
- **Coupure réseau :** Si le câble est débranché, TCP tente des retransmissions. Si la coupure est brève, la connexion survit. Sinon, une erreur `ETIMEDOUT` est retournée.

4 Partie 4 : Transfert de messages en mode UDP

4.1 Comparaison TCP vs UDP

Contrairement à TCP, UDP est un protocole sans état. Nous avons implémenté un client capable de détecter une absence de réponse via un timeout (`SO_RCVTIMEO`).

Caractéristique	**TCP**	**UDP**
Connexion	Oui	Non
Fiabilité	Garantie	Non garantie
Vitesse	Plus lent	Plus rapide

Table 1 – Comparaison des protocoles de transport

5 Partie 5 : Serveur en mode concurrent

5.1 Architecture

Nous avons réalisé un serveur multi-services utilisant `select()` pour multiplexer l'écoute sur les ports 5000, 5001 et 5002, et `fork()` pour la concurrence.

```
while (1) {
    select(maxfd + 1, &readfds, NULL, NULL, NULL);
    for (int i = 0; i < NB_SERVICES; i++) {
        if (FD_ISSET(listen_fds[i], &readfds)) {
            int conn_fd = accept(listen_fds[i], ...);
            if (fork() == 0) {
                // Processus fils
                close_listening_sockets();
                traiter_service(i, conn_fd);
                exit(0);
            }
            close(conn_fd); // Le pere ferme sa copie
        }
    }
}
```

5.2 Services proposés

1. **Port 5000 :** Envoi de l'heure (60 messages).
2. **Port 5001 :** Nombre de processus (`ps aux | wc -l`).
3. **Port 5002 :** Echo interactif.

6 Conclusion

Ce TP nous a permis de comprendre les fondamentaux de la programmation réseau. La maîtrise de `select()` et `fork()` est cruciale pour construire des serveurs robustes et performants capables de servir une multitude de clients simultanément.